Resume Bullet Points

Ride-Sharing Backend Project | Spring Boot · Kafka · Redis · Stripe · Microservices

3 role-specific sections below. Each bullet shows the exact text to put on your resume, followed by a shaded explanation of WHY it matters and what you should be ready to discuss in the interview.

Project:	Ride-Sharing Backend — Production-Grade Microservices System
Stack:	Spring Boot 3.2 · Apache Kafka · Redis · PostgreSQL · Stripe · Docker · Eureka · Zipkin · Prometheus
GitHub:	github.com/yourusername/ridesharing-backend

Role 1 — Backend Developer

Spring Boot | **Kafka** | **Redis** | **PostgreSQL** | **Stripe** | **Microservices** | **JWT** | **Docker**

■ Designed and implemented a production-grade ride-sharing backend using Spring Boot 3.2 with 7 independent microservices (User, Driver, Ride, Pricing, Payment, Notification, API Gateway) following the database-per-service pattern

> *WHY IT MATTERS: Shows you can design real systems, not toy projects. 'Database-per-service' is a key microservices pattern that interviewers often ask about. WHAT TO SAY: Each service owns its own PostgreSQL schema — this prevents tight coupling and lets each service evolve its data model independently without affecting others.*

■ Built real-time driver geo-matching using Redis GeoSet (GEORADIUS), enabling sub-millisecond lookup of nearest available drivers within a configurable radius with O(log N) query complexity

> *WHY IT MATTERS: Geospatial matching is a core Uber/Ola feature — showing you built it from scratch impresses interviewers. WHAT TO SAY: Redis stores driver GPS as longitude/latitude in a sorted set. GEORADIUS searches by distance using the Haversine formula internally. We use it because a standard SQL 'distance < X' query becomes a full table scan at scale, while Redis GeoSet is O(log N).*

■ Implemented event-driven architecture with Apache Kafka — 7 topics coordinating ride lifecycle events across 5 consumer services, with idempotent producers (acks=all, enable.idempotence=true) for zero data loss

> *WHY IT MATTERS: Kafka is asked about in almost every senior backend interview. WHAT TO SAY: acks=all means the broker waits for all in-sync replicas to acknowledge before confirming — this prevents data loss if the leader crashes. enable.idempotence=true ensures even retried messages are not duplicated. We use Kafka so services are decoupled — ride-service doesn't need to know which services are listening.*

■ Integrated Stripe Payment Gateway with async webhook handling, Resilience4j circuit breaker (opens at 50% failure rate), and two-layer idempotency (DB dedup + Stripe idempotency key) preventing double-charging

> *WHY IT MATTERS: Payment reliability is critical — double-charging users is a P0 bug. WHAT TO SAY: Layer 1 — we check DB before processing: if a payment record for this rideId already exists, we skip. Layer 2 — we send Stripe an idempotency key (rideId) so even if Stripe receives the request twice, it returns the same PaymentIntent. The circuit breaker opens after 50% failure rate for 30s to prevent cascading failures to Stripe's API.*

■ Engineered dynamic surge pricing with Redis-backed demand/supply counters per geo-zone, recalculated every 30s via @Scheduled jobs, with multiplier capped at 3x — served via REST API with sub-10ms latency

> *WHY IT MATTERS: Surge pricing is a classic system design question — you now have a real implementation to talk about. WHAT TO SAY: We increment demand counter when a ride is requested in a zone, supply counter when a driver goes online. Every 30 seconds a @Scheduled job reads both counters and updates a surge multiplier key in Redis. The API reads the cached multiplier — no DB query, hence sub-10ms response.*

■ Implemented JWT-based stateless authentication at API Gateway level using Spring Cloud Gateway GlobalFilter with BCrypt hashing and role-based header forwarding to downstream services

> *WHY IT MATTERS: Security is asked in every backend interview. WHAT TO SAY: The JWT contains userId and role — signed with HS256. The API Gateway validates the signature on every request (no DB call needed — stateless). After validation, it injects X-User-Id and X-User-Role headers. Downstream services trust these headers instead of re-validating — this centralises auth logic.*

■ Set up full observability stack: Zipkin distributed tracing across all services, Prometheus metrics scraping, Grafana dashboards for JVM/Kafka/Redis monitoring, and structured logging with trace ID correlation

> *WHY IT MATTERS: Senior roles expect you to know how to debug production issues. WHAT TO SAY: In a microservices system, a single request touches 3-4 services. Zipkin gives each request a traceId that flows through all services — so when something fails you can see exactly which service/method was slow. Prometheus scrapes /actuator/prometheus every 15s. Grafana dashboards show JVM heap, Kafka consumer lag, Redis memory.*

■ Containerised entire platform with Docker Compose — all 9 services (including Kafka, Redis, PostgreSQL, Eureka, Zipkin) spin up with a single command for consistent dev/prod environments

> *WHY IT MATTERS: Docker is expected at every company. WHAT TO SAY: docker-compose up -d starts Kafka, Zookeeper, Redis, 4 PostgreSQL databases, Eureka, Zipkin, Prometheus, Grafana, and all 7 Spring Boot services in correct dependency order using healthchecks. This eliminates 'it works on my machine' problems and mirrors production infrastructure.*

***PRO TIP:** For backend roles, focus on Kafka reliability guarantees, Redis GeoSet choice, and payment idempotency. These 3 topics alone will take up 30+ minutes of a typical backend interview.*

Role 2 — Full Stack Developer

Spring Boot	REST APIs	Kafka	Redis	Stripe	JWT	Docker	Microservices

■ **Built a scalable ride-sharing platform backend with 7 Spring Boot microservices exposing 20+ REST API endpoints consumed by mobile/frontend clients, with request routing via Spring Cloud Gateway**

> *WHY IT MATTERS: Full-stack roles want you to understand the full request flow. WHAT TO SAY: The API Gateway is the single entry point for frontend/mobile — it handles routing, JWT auth, and rate limiting. All 20+ endpoints follow RESTful conventions (GET for reads, POST for creates, PUT for updates). Service-to-service calls happen via Kafka events, not HTTP, to avoid tight coupling.*

■ **Designed JWT-based stateless authentication flow — login returns a signed token, API Gateway validates it on every request and injects user context headers, eliminating session management overhead**

> *WHY IT MATTERS: Every app needs auth — showing you designed a proper stateless flow matters. WHAT TO SAY: Login POST /api/users/login returns a JWT with userId + role. Every subsequent request must include Authorization: Bearer . The Gateway's JwtAuthFilter validates it without hitting the DB — this means the auth layer scales horizontally without a shared session store.*

■ **Implemented real-time driver location tracking with Redis GeoSet — driver GPS updated every 5 seconds, nearest-driver queries return results in under 5ms to support seamless booking UX**

> *WHY IT MATTERS: Real-time features are a differentiator in full-stack interviews. WHAT TO SAY: The driver app sends POST /api/drivers/location every 5 seconds. This updates the Redis GeoSet for that driver. When a rider books, we query Redis for drivers within 5km — results come back in under 5ms. No PostgreSQL is involved in the hot path — Redis handles it entirely.*

■ **Integrated Stripe payment processing with webhook support for async payment confirmation and automatic retry logic, ensuring reliable end-to-end payment flow from ride completion to rider receipt**

> *WHY IT MATTERS: Payments are the most business-critical feature in any product. WHAT TO SAY: Payment is triggered asynchronously — when Kafka fires a ride.completed event, payment-service creates a Stripe PaymentIntent. Stripe calls our /webhook endpoint to confirm success or failure. If the webhook fails, Stripe retries up to 3 days — our idempotency key prevents double-charges on retry.*

■ **Built dynamic surge pricing REST API (GET /api/pricing/estimate) returning real-time fare estimates with surge multiplier, distance, and duration — powered by Redis-cached demand/supply ratios per city zone**

> *WHY IT MATTERS: Shows you can build product features that are both technically complex and user-facing. WHAT TO SAY: The frontend calls /api/pricing/estimate with pickup and dropoff coordinates. We calculate distance using the Haversine formula, fetch the zone's surge multiplier from Redis (updated every 30s), and return base fare + surge adjustment. The entire API call takes under 20ms.*

■ **Implemented full ride lifecycle state machine (REQUESTED → DRIVER_ASSIGNED → STARTED → COMPLETED) with status-change events triggering real-time push notifications to rider and driver apps**

> *WHY IT MATTERS: State machines come up in every system design interview — you have a real implementation. WHAT TO SAY: Each ride has a RideStatus enum. Transitions are validated — you can't go from REQUESTED to COMPLETED directly. Each transition publishes a Kafka event. Notification-service listens to all ride.* topics and sends push notifications to the relevant user IDs.*

■ **Configured service discovery (Eureka) and load-balanced routing (Spring Cloud LoadBalancer) enabling zero-downtime rolling deployments and horizontal scaling of any individual service**

> *WHY IT MATTERS: Production systems need dynamic service discovery — hardcoded URLs break when you scale. WHAT TO SAY: Every service registers with Eureka on startup with its hostname and port. Spring Cloud LoadBalancer reads the Eureka registry and round-robins requests across multiple instances. This means you can spin up 3 instances of ride-service without changing any config.*

■ **Dockerised entire platform with Docker Compose — reproducible environment setup for development, testing, and deployment with all dependencies (Kafka, Redis, PostgreSQL) pre-configured**

> *WHY IT MATTERS: Full-stack roles want engineers who own their deployments. WHAT TO SAY: The docker-compose.yml defines all 9 services with correct startup order (Zookeeper before Kafka, PostgreSQL before Spring services). Each service has health checks. Environment variables are externalized — swap .env for prod secrets in CI/CD. This makes onboarding a new developer a single-command operation.*

***PRO TIP:** For full-stack roles, be ready to draw the request lifecycle: Browser/App → Gateway → Service → Kafka → DB. Also practice explaining the JWT flow without looking at code.*

Role 3 — Software Engineer (General / Product Companies)

System Design | Distributed Systems | Kafka | Redis | Microservices | Resilience4j | Docker

■ **Architected a production-grade distributed system for a ride-sharing platform with 7 independently deployable microservices — demonstrating expertise in distributed systems design, fault tolerance, and scalability**

> *WHY IT MATTERS: Product companies (Swiggy, Zomato, Flipkart, FAANG) hire for distributed systems thinking. WHAT TO SAY: Each service runs in its own process with its own DB — they communicate via Kafka events (async) or via REST through the API Gateway (sync). This means failure in one service (e.g. notification) doesn't bring down the booking flow. You can also scale each service independently based on load.*

■ **Applied choreography-based Saga pattern for distributed transaction management across ride lifecycle and payment flow — avoiding two-phase commit while ensuring eventual consistency across services**

> *WHY IT MATTERS: Distributed transactions are a top-5 system design interview topic. WHAT TO SAY: In a monolith you'd use DB transactions. In microservices, each service has its own DB — you can't do a single ACID transaction. Saga pattern coordinates this: each service does its local transaction and publishes an event. If payment fails, ride-service listens to payment.failed and cancels the ride (compensating transaction). No 2PC locks needed.*

■ **Solved core distributed systems challenges: exactly-once Kafka delivery, Redis TTL-based automatic expiry, geospatial nearest-neighbour queries at scale, and idempotent payment processing**

> *WHY IT MATTERS: Shows you understand the hard parts — not just how to set up services but how to handle failure. WHAT TO SAY: Kafka exactly-once = idempotent producer + transactional consumer. Redis TTL = ride requests auto-expire after 60s so stale requests aren't accepted. GEORADIUS = O(log N) geo queries without SQL. Idempotent payments = DB check + Stripe idempotency key prevents double-charge.*

■ Designed for resilience: Resilience4j circuit breakers on external Stripe API (opens at 50% failure rate, 30s recovery), Kafka dead-letter topics for failed events, and automatic retry with exponential backoff

> *WHY IT MATTERS: Fault tolerance separates good engineers from great ones. Interviewers probe this deeply. WHAT TO SAY: Circuit breaker: if Stripe fails 5 out of 10 times (50%), we open the circuit — subsequent calls immediately return the fallback (mark payment as PENDING, retry later) instead of hammering a failing service. This prevents cascading failures. After 30s, we allow one test request through — if it succeeds, circuit closes.*

■ Implemented event-driven microservices with Apache Kafka — loose coupling between 7 services means any service can fail/scale independently without affecting others, enabling zero-downtime deployments

> *WHY IT MATTERS: Event-driven architecture is a fundamental pattern at scale. WHAT TO SAY: ride-service publishes to ride.requested. It doesn't know or care who is listening. driver-service, pricing-service, and notification-service all consume independently. If notification-service is down for 5 minutes, its Kafka consumer group lag builds up — when it comes back, it processes all missed events. No data is lost.*

■ Built real-time geospatial matching using Redis GeoSet with O(log N) nearest-neighbour queries, handling high-frequency GPS updates (every 5s) from thousands of concurrent drivers efficiently

> *WHY IT MATTERS: Geospatial systems are asked about in design rounds for Uber, Ola, Rapido. WHAT TO SAY: Redis GEOADD stores driver locations as (longitude, latitude) in a sorted set encoded with a 52-bit geohash. GEORADIUS queries by center + radius using the geohash index. For 10,000 concurrent drivers, each GPS update is O(log N) = ~14 operations. Compare to SQL: a full table scan or manual bounding box query.*

■ Demonstrated production engineering practices: centralised config (Spring Cloud Config), service discovery (Eureka), distributed tracing (Zipkin), metrics dashboards (Prometheus/Grafana), full containerisation

> *WHY IT MATTERS: Shows you think about operations, not just feature delivery. WHAT TO SAY: Spring Cloud Config Server serves application.yml from a central location — change a config and all services pick it up without redeploying. Zipkin tracing shows the full journey of a request across services with timing. Prometheus + Grafana alert on anomalies like rising Kafka consumer lag or Redis memory approaching limit.*

■ Ensured payment reliability with two-layer idempotency (DB-level deduplication + Stripe idempotency keys), preventing double-charges even under Kafka message redelivery or network retries

> *WHY IT MATTERS: Payment correctness is a P0 problem — every fintech/product company cares deeply about this. WHAT TO SAY: Kafka guarantees at-least-once delivery — the same ride.completed event can arrive twice if the consumer crashes mid-processing. Layer 1: we check if a Payment row with this rideId already exists before doing anything. Layer 2: we send rideId as the Stripe idempotency key — Stripe returns the same response for duplicate requests within 24h.*

***PRO TIP:** For SWE roles at Swiggy/Zomato/Flipkart/FAANG, lead with system design decisions. Know the trade-offs: Kafka vs REST, Saga vs 2PC, Redis GeoSet vs PostGIS, stateless vs stateful auth.*

How to Use These Bullet Points

Pick 4-5 bullets	Don't copy all 8. Pick the ones that match the job description keywords best. If JD says 'event-driven', pick that bullet and lead with it.
Add your own numbers	Replace placeholders with real values if you have them. '7 microservices', 'sub-5ms Redis queries', '60s TTL expiry' are already specific — keep those.
Match JD language exactly	If the JD says 'distributed systems', use that phrase. If it says 'message queue', use that. ATS systems keyword-match.
STAR format for interviews	Each bullet = a story. Situation: ride-sharing app. Task: match drivers in real-time. Action: Redis GeoSet. Result: sub-5ms queries. Practice saying each one aloud.
GitHub link is mandatory	Push the project to GitHub with a clean README. Interviewers WILL look at your code before or during the interview. Uncommented code = red flag.

All bolded technologies are covered in depth in the Interview-QnA.pdf